

Tutorial 06: Specular Highlights, Indexed Geometry, Directional Lighting - Teaching Guide

Overview

This tutorial completes the **Phong lighting model** by adding **specular highlights** (shininess). It also introduces two important rendering techniques: **per-fragment lighting** (moving calculations from the vertex shader to the fragment shader for smoother results) and **indexed geometry rendering** (`glDrawElements`) which efficiently reuses shared vertices.

Prerequisites and Setup

```
.\shader_tutorial_venv\Scripts\Activate.ps1
cd tutorials
python 06_SpecularBlinnPhong.py
```

Expected Output: A dark sphere with a small, sharp yellow-green specular highlight (the "shiny spot") rendered with smooth per-pixel lighting. The sphere uses a procedurally generated UV sphere with indexed triangle rendering.

Interactive Controls:

- **ESC** - Exit
-

Learning Objectives

By the end of this tutorial, students will understand:

1. What specular highlights are physically and mathematically
 2. The difference between Phong and Blinn-Phong specular methods
 3. How the "half-vector" shortcut works
 4. Why per-fragment lighting produces better results than per-vertex lighting
 5. Indexed geometry with `glDrawElements` and `GL_ELEMENT_ARRAY_BUFFER`
 6. How two VBOs can be bound simultaneously to different targets
 7. How to procedurally generate sphere geometry
-

What's New in This Tutorial

Compared to Tutorial 05:

- ☒ **Specular highlights** — Blinn-Phong half-vector method
 - ☒ **Per-fragment lighting** — fragment shader does the heavy lifting
-

- ☒ **phong_weightCalc** returns **vec2** — diffuse AND specular weights
- ☒ **glDrawElements** — indexed draw call
- ☒ **Two VBO targets** — **GL_ARRAY_BUFFER** + **GL_ELEMENT_ARRAY_BUFFER**
- ☒ **Procedural sphere** — UV sphere with 8-float interleaved vertex records
- ☒ **3 new uniforms** — **Light_specular**, **Material_specular**, **Material shininess**

The Complete Phong Model

Final Colour = Ambient + Diffuse + Specular

Ambient = (Global_ambient + Light_ambient) * Material_ambient

Diffuse = Light_diffuse * Material_diffuse * max(0, dot(N, L))

Specular = Light_specular * Material_specular * pow(dot(H, N), shininess)

Where:

N = surface normal (normalised)

L = light direction (normalised)

H = half-vector (normalised, Blinn-Phong only)

shininess = sharpness exponent

Visual — the three components:

Ambient only:

•
(flat grey)

Ambient + Diffuse:

•
(shaded sphere)

Full Phong (+ Specular):

● ← specular spot
(shaded + highlight)

Code Breakdown

1. Specular Highlights — What Are They?

A shiny surface reflects light **in a specific direction**. When the reflected ray aligns with the viewer's direction, you see a bright "hot spot".

Physical analogy:

- Matte chalk: diffuse only (scattered everywhere)
- Billiard ball: mostly diffuse + a sharp specular highlight
- Mirror: nearly all specular (perfectly reflective)

Phong's original formula:

```
R = reflect(L, N) // exact reflection of light around normal
Spec_factor = pow(dot(R, V), shininess)
```

```
// V = direction from surface to eye
// R = direction of reflected light ray
```

Problem: `reflect()` and `dot(R, V)` require recalculating `R` for every fragment.

2. Blinn-Phong — The Half-Vector Shortcut

```
// In eye-space: eye is always at (0,0,0), camera points down -Z
// Eye direction (constant in eye-space):
//   eye_dir = (0, 0, -1)

vec3 Light_half = normalize(EC_Light_location - vec3(0.0, 0.0, -1.0));
```

The idea: Instead of computing the reflection ray, compute the **half-vector** `H` — the vector that bisects the angle between the light direction and the view direction:

```
H = normalize(L + V)
```

In eye-space, `V = (0, 0, -1)` (constant!), so `H` needs to be computed only **once per draw call** for a directional light.

```
Blinn-Phong: Spec_factor = pow(dot(H, N), shininess)
```

Why Blinn over Phong?

Aspect	Phong	Blinn-Phong
Reflection vector	Needed — expensive	Not needed
Constant per-frame	No	Yes (directional light + eye-space)
Physical accuracy	Approximate	Closer to real materials
Shininess range	Need ~4× higher for same look	Lower values, more intuitive
GPU hardware support	Less common	Used in legacy GL built-ins

3. Updated `phong_weightCalc` Returns `vec2`

```
vec2 phong_weightCalc(
    in vec3 light_pos,    // normalised light direction
    in vec3 half_light,   // half-way vector
    in vec3 frag_normal,  // normalised surface normal
```

```

    in float shininess
) {
    float n_dot_pos = max( 0.0, dot( frag_normal, light_pos ) );
    float n_dot_half = 0.0;
    if (n_dot_pos > -0.05) {
        n_dot_half = pow( max(0.0, dot(half_light, frag_normal)), shininess );
    }
    return vec2( n_dot_pos, n_dot_half );
}

```

New vs Tutorial 05:

- Now takes two extra parameters: `half_light` and `shininess`
- Returns `vec2` — `.x` = diffuse weight, `.y` = specular weight
- `n_dot_half` only calculated when `n_dot_pos > -0.05` (surface faces the light)
 - Without this guard, you'd see a specular halo on the dark side of the sphere

Usage in fragment shader:

```

vec2 weights = phong_weightCalc(EC_Light_location, Light_half, frag_normal,
Material_shininess);
// weights.x = diffuse multiplier
// weights.y = specular multiplier

```

4. Per-Fragment Lighting

Tutorial 05: Per-vertex lighting

```

Vertex 0 → colour A
Vertex 1 → colour B      ← calculated in vertex shader
Vertex 2 → colour C

Rasteriser interpolates A, B, C across the triangle pixels

```

Tutorial 06: Per-fragment lighting

```

Vertex 0 → normal N0
Vertex 1 → normal N1      ← only normals passed as varyings
Vertex 2 → normal N2

Rasteriser interpolates N0, N1, N2 → per-pixel normal N_px
Fragment shader: colour = lighting(N_px) ← calculated per pixel

```

Why does this matter for specular?

Problem with per-vertex specular:

If the specular highlight falls BETWEEN two vertices,
it gets averaged out or disappears completely.

Per-fragment specular:

Every pixel has its own normal → the highlight is always visible
and always smooth, regardless of polygon density.

Vertex shader (very simple now):

```
attribute vec3 Vertex_position;
attribute vec3 Vertex_normal;
varying vec3 baseNormal;

void main() {
    gl_Position = gl_ModelViewProjectionMatrix * vec4(Vertex_position, 1.0);
    baseNormal = gl_NormalMatrix * normalize(Vertex_normal);
}
```

Only transforms position and passes the normal as a varying. All lighting moved to fragment shader.

Fragment shader does the lighting:

```
varying vec3 baseNormal;

void main() {
    vec3 frag_normal = normalize(baseNormal); // re-normalise after
    interpolation!
    vec3 EC_Light = normalize(gl_NormalMatrix * Light_location);
    vec3 Light_half = normalize(EC_Light - vec3(0, 0, -1));
    vec2 weights = phong_weightCalc(EC_Light, Light_half, frag_normal,
    Material_shininess);
    gl_FragColor = clamp(
        ambient + diffuse * weights.x + specular * weights.y,
        0.0, 1.0
    );
}
```

Important: `normalize(baseNormal)` in the fragment shader!

When the rasteriser interpolates two unit vectors, the result is NOT guaranteed to be unit length. Always re-normalise in the fragment shader.

5. Sphere Geometry — Procedural Generation

```
def generate_sphere(radius=1.0, stacks=24, slices=24):
    # Returns (coords_array, indices_array)
```

A **UV sphere** is generated by:

1. Dividing the sphere into horizontal rings (stacks)
2. Each ring is divided into vertical segments (slices)
3. Vertices are at the intersections: $(x, y, z) = (r \cdot \cos(\theta) \cdot \cos(\phi), r \cdot \sin(\theta) \cdot \cos(\phi), r \cdot \sin(\theta) \cdot \sin(\phi))$

For a unit sphere: normal = position vector (surface point on a sphere centred at origin is always the outward normal).

Vertex record layout (8 floats = 32 bytes):

```
[pos.x, pos.y, pos.z, u, v, norm.x, norm.y, norm.z]
  ↑ 0 bytes      ↑ 12 bytes  ↑ 20 bytes
  position      tex coords  normal
```

Texture coordinates (u, v) are included for stride compatibility with OpenGLContext's Sphere class. We skip them for now (no texture tutorial yet).

6. Indexed Geometry — `glDrawElements`

The problem with vertex arrays (`glDrawArrays`):

A cube has 8 unique vertices but 12 triangles = 36 vertex slots. Each vertex is duplicated ~4.5 times. For a sphere with 1000 vertices and 2000 triangles, that's 6000 VBO entries — 6× the needed data!

Solution: Index buffers

Store vertices once, then list which vertices make each triangle:

```
Vertex buffer:      Index buffer:
[V0, V1, V2, V3, V4] [0, 1, 2, 0, 2, 3, 1, 4, 2, ...]
                    ↑ tri 1  ↑ tri 2  ↑ tri 3

Triangle 1 uses V0, V1, V2
Triangle 2 uses V0, V2, V3 (V0 and V2 shared with triangle 1!)
```

Memory savings: For a typical mesh, indices use ~1% the memory of duplicated vertices.

7. Two VBO Targets

```
self.coords = vbo.VBO(coords_data, target=GL_ARRAY_BUFFER)
self.indices = vbo.VBO(indices_data, target=GL_ELEMENT_ARRAY_BUFFER)
```

```
self.coords.bind() # binds to GL_ARRAY_BUFFER
self.indices.bind() # binds to GL_ELEMENT_ARRAY_BUFFER
```

OpenGL VBO targets:

Target	Purpose	Used by
GL_ARRAY_BUFFER	Per-vertex attribute data	glVertexAttribPointer
GL_ELEMENT_ARRAY_BUFFER	Index data (triangle vertex indices)	glDrawElements

These are **independent bind points** — binding to one does NOT affect the other. Both can be bound at the same time. This is why the second `bind()` doesn't overwrite the first.

Must unbind both:

```
finally:
    self.coords.unbind() # frees GL_ARRAY_BUFFER
    self.indices.unbind() # frees GL_ELEMENT_ARRAY_BUFFER
```

8. glDrawElements Call

```
glDrawElements(
    GL_TRIANGLES,      # primitive type
    self.count,        # number of indices to read
    GL_UNSIGNED_SHORT, # data type of each index value
    self.indices        # pointer (reads from GL_ELEMENT_ARRAY_BUFFER)
)
```

Parameter guide:

Parameter	Meaning
mode	GL_TRIANGLES, GL_LINES, etc.
count	Total number of indices (e.g., 2000 triangles × 3 = 6000)
type	GL_UNSIGNED_BYTE (0-255), GL_UNSIGNED_SHORT (0-65535), GL_UNSIGNED_INT (0-4billion)
indices	NULL or offset if index VBO is bound; pointer to array otherwise

Why `GL_UNSIGNED_SHORT`?

- Short = 2 bytes per index, supports up to 65,535 unique vertices
- Our sphere has ~1000 vertices — short is sufficient
- Use `GL_UNSIGNED_INT` for meshes with >65K vertices

`glDrawArrays` vs `glDrawElements`:

	<code>glDrawArrays</code>	<code>glDrawElements</code>
Reads	Vertices sequentially from VBO	Vertices in the order the index buffer specifies
Shared vertices	Not possible (must duplicate)	Yes — vertex used by many triangles stored once
Memory use	Higher (duplicated)	Lower (shared vertices)
Performance	Simpler	Better for typical meshes

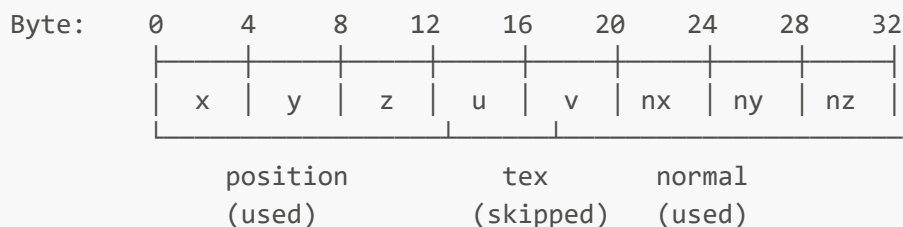
9. Vertex Record Layout and Stride

```
stride = coords_data.shape[1] * 4 # 8 floats × 4 bytes = 32
```

```
# Position: 3 floats at offset 0
glVertexAttribPointer(self.Vertex_position_loc, 3, GL_FLOAT, False, stride,
self.coords)

# Normal: 3 floats at offset 5*4 = 20 bytes (skip x,y,z,u,v)
glVertexAttribPointer(self.Vertex_normal_loc, 3, GL_FLOAT, False, stride,
self.coords + (5*4))
```

Memory layout:



The texture coordinates are in the VBO but we skip them (+ `5*4` offset jumps past them).

Shininess Parameter — Effect on Highlight Size

```
glUniform1f(self.Material_shininess_loc, 0.95)
```

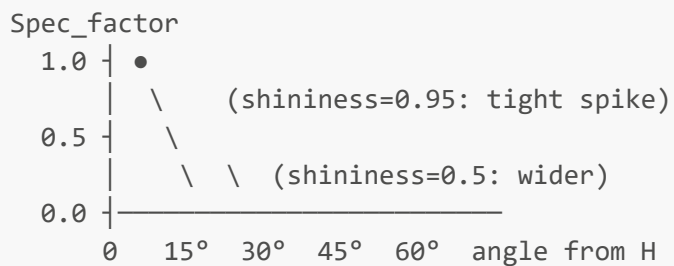
The shininess exponent controls how sharp/spread the specular highlight is:

```
Spec_factor = pow(dot(H, N), shininess)
```

shininess	Highlight	Real-world material
0.1	Very large, diffuse	Rough plastic
0.5	Medium spread	Painted surface
0.9	Small, sharp	Polished metal
0.95	Very small, intense (this tutorial)	Wet surface

Note: In Blinn-Phong, shininess values are lower than Phong for the same visual result. In legacy OpenGL, shininess was in the range 0-128. In normalised [0,1] form (as used here), 0.95 is very high.

Graph:



Complete Data Flow — Per-Fragment Lighting

VBO data (one vertex):

```
[0.0, 1.0, 0.0, 0.5, 0.0, 0.0, 1.0, 0.0]
  ← position →  ← uv →      ← normal →
(top of sphere) (points upward)
```

Vertex Shader:

```
1. gl_Position = MVP * (0, 1, 0, 1)
2. baseNormal = NormalMatrix * normalize(0, 1, 0) → eye-space normal
```

Rasterisation: `baseNormal` interpolated across triangle pixels.

Fragment Shader (one pixel near the top of sphere):

```
1. frag_normal = normalize(baseNormal) ← re-normalise!
2. EC_Light = normalize(NormalMatrix * (6, 2, 4)) → eye-space light
3. Light_half = normalize(EC_Light - (0, 0, -1)) → half-vector
4. weights = phong_weightCalc(EC_Light, Light_half, frag_normal, 0.95)
   → weights.x = 0.8 (diffuse: surface faces light)
   → weights.y = 0.9 (specular: near highlight centre)
5. gl_FragColor = clamp(
    (0.05,0.05,0.05,1)*(0.1,0.1,0.1,1) ← global ambient
    + (0.1,0.1,0.1,1)*(0.1,0.1,0.1,1) ← light ambient
    + (0.25,0.25,0.25,1)*(0.15,0.15,0.15,1)*0.8 ← diffuse
    + (0.0,1.0,0.0,1)*(1,1,1,1)*0.9, ← YELLOW specular!
    0, 1)
   → (0.0, 0.9+, ...) → bright yellow-green pixel
```

Visual Comparison: Per-Vertex vs Per-Fragment Lighting

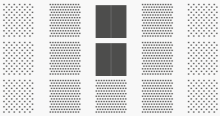
Per-Vertex (Tutorial 05 approach on sphere):

- 100 vertices on sphere
- Specular calculated at each vertex
- Highlight may look jagged or miss the "sweet spot"

```
[ ] [ ] [S] [ ] ← S = vertex with specular contribution
 \ / \ / \ / \
  *  *  *  * ← interpolated colours
```

Per-Fragment (Tutorial 06):

- 100 vertices, but 10,000+ pixels
- Specular calculated per pixel
- Smooth, accurate highlight regardless of vertex density



← smooth gradient per pixel

Common Student Questions

Q: When should I use per-vertex vs per-fragment lighting?

A: Use per-fragment when you need accurate specular highlights or any effect that requires precise per-pixel information. Use per-vertex for ambient and diffuse-only lighting on high-poly meshes to save fragment shader cost.

Q: Why do I need to `normalize(baseNormal)` in the fragment shader if it was already normalised in the vertex shader?

A: Linear interpolation of unit vectors produces a vector slightly shorter than 1.0 (the interpolation "cuts corners"). Without re-normalising, the dot products will be slightly off, causing subtle shading errors.

Q: What's the `u`, `v` data in the sphere VBO that we're ignoring?

A: Texture coordinates — they map the sphere surface to a 2D image. We include them to match the data layout (32 bytes stride) but won't use them until a texturing tutorial.

Q: Can I move the specular highlight?

A: Yes — change `Light_location`:

```
glUniform3f(self.Light_location_loc, -6.0, 2.0, 4.0) # light from the left
```

Q: Why is the sphere dark but has a small bright spot?

A: The material diffuse and ambient values are deliberately very low (`0.15`, `0.1`) so the specular highlight dominates visually. Increase `Material_diffuse` for a brighter sphere.

Q: What's the difference between `Light_specular` and `Material_specular`?

A: Both multiply together. `Light_specular` is the colour of the light's specular contribution (yellow here). `Material_specular` is the material's specular reflectance (white = reflects all specular colours). `Material_specular = (1,0,0)` would make the highlight red regardless of `Light_specular`.

Experiments to Try

1. Change Shininess

```
glUniform1f(self.Material_shininess_loc, 0.1) # very dull, spread highlight  
glUniform1f(self.Material_shininess_loc, 0.99) # razor-sharp highlight
```

2. Coloured Specular

```
glUniform4f(self.Light_specular_loc, 1.0, 0.0, 0.0, 1.0) # red highlight  
glUniform4f(self.Material_specular_loc, 0.0, 0.0, 1.0, 1.0) # blue material  
spec
```

3. Brighter Diffuse Sphere

```
glUniform4f(self.Material_diffuse_loc, 0.6, 0.6, 0.6, 1.0)  
glUniform4f(self.Light_diffuse_loc, 1.0, 1.0, 1.0, 1.0)
```

Effect: Visible diffuse shading around the whole sphere, not just the specular spot.

4. Rotate the Sphere (animate)

```
# In Render(), before glUseProgram:
glRotatef(self._angle, 0, 1, 0)
self._angle += 0.5
```

Effect: Specular highlight stays fixed (light is world-space), sphere rotates under it.

5. Multiple Lights

```
// In fragment shader, repeat for a second light:
vec3 EC_Light2 = normalize(gl_NormalMatrix * Light2_location);
vec3 Half2 = normalize(EC_Light2 - vec3(0,0,-1));
vec2 w2 = phong_weightCalc(EC_Light2, Half2, frag_normal, Material_shininess);
gl_FragColor = clamp(ambient + diffuse*w.x + spec*w.y + diffuse2*w2.x +
spec2*w2.y, 0, 1);
```

Debugging Tips

Problem: Specular highlight not visible

- Check `Material_specular` is not `(0,0,0,0)`
- Check `Light_specular` is not `(0,0,0,0)`
- Try lowering `Material_shininess` to 0.3 (larger highlight, easier to spot)
- Make sure `n_dot_half` guard condition isn't blocking it

Problem: Specular appears on the wrong side (back of sphere)

- The `n_dot_pos > -0.05` guard should prevent this
- Check normals are pointing outward (for a sphere, `normal == position direction`)

Problem: Faceted/hard-edged shading on sphere

- Increase stacks and slices in `generate_sphere()`
- Make sure normals are per-vertex averages, not per-face

Problem: Index buffer out of bounds

- Check `GL_UNSIGNED_SHORT` vs `GL_UNSIGNED_INT` matches your index data type
- Verify max index value < number of vertices

Visualise normals as colours:

```
gl_FragColor = vec4(normalize(baseNormal) * 0.5 + 0.5, 1.0);
```

Performance Considerations

Technique	Cost	Benefit
Per-fragment lighting	~10-100× more fragment invocations than vertex	Accurate specular, smooth shading
Indexed rendering	Small overhead for index lookup	3-6× memory reduction, better GPU cache use
Sphere at 32×32 stacks/slices	~1100 vertices, ~6200 indices	Smooth with reasonable vertex count
<code>normalize()</code> in fragment shader	Small per-pixel cost	Required for correctness

Real-world tip: Modern games use per-fragment lighting everywhere. The cost is offset by techniques like:

- Level-of-detail (simpler mesh when far away)
- Deferred shading (calculate lighting only for visible pixels)
- Clustered lighting (efficiently handle many lights)

Summary Diagram



```
1. Re-normalise baseNormal
2. Compute eye-space light direction
3. Compute half-vector
4. phong_weightCalc() → (diffuse, specular)
5. gl_FragColor = ambient + diff + spec
```

↓
Screen (smooth sphere with specular highlight!)

Complete Tutorial Series Summary

Tutorial 01: Basic VBO + shader, green triangle
Tutorial 02: Varying values, per-vertex colours
Tutorial 03: Uniforms, fog effect
Tutorial 04: Multiple attributes, tween animation
Tutorial 05: Normals, ambient + diffuse lighting
Tutorial 06: Specular highlights, per-fragment, indexed geometry

The three data channels mastered:

- **Attributes** → per-vertex data from VBO (position, normal, colour, UV...)
- **Uniforms** → per-draw-call constants from CPU (light position, material colour...)
- **Varyings** → vertex → fragment interpolated values (normals, colours, UVs...)

Future topics unlocked:

- Textures (UV coordinates + sampler uniforms)
- Multiple lights (loop in fragment shader)
- Vertex Array Objects (VAOs) — bundle all attribute state
- Normal maps (per-fragment normal from texture)
- Shadow mapping
- Post-processing effects (render to texture, then process)